

Examen-SO-Modulo-II-2024-Luis-Es...



Darigod



Sistemas Operativos



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada**



[Accede al documento original](#)



Escuela de
Organización
Industrial

Contigo que evolucionas.
Contigo que lideras. Contigo que transformas.

**Esto es EOI.
Mismo propósito,
nueva energía.**



Descubre más aquí



EOI Escuela de
Organización
Industrial

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali oohh
esto con 1 coin me
lo quito yo...

WUOLAH

Examen Módulo 2

Profesor: Luis López Escudero (23-24, Grupo A3)

Ejercicio 1 – sumador

Crear el programa *sumador*:

Sintaxis: `sumador <dir>`

Parámetros:

- `<dir>` Directorio donde queremos ejecutar el programa sumador.

Funcionamiento:

- El programa leerá de la STDIN un número octal de 3 dígitos. Este número representa una máscara de **permisos**, los cuales incluyen los permisos de propietario, grupo y otros.
- Si el número octal leído es distinto de 0, el programa iterará por cada una de las entradas del directorio `<dir>` pasado como argumento.
 - Para cada una de las entradas, si se cumple que es un **archivo regular** y tiene **los mismos permisos**, el programa imprimirá por la STDERR información del archivo, con el siguiente formato:

Archivo: <nombre del archivo> | Bloques: <número de bloques> | Tamaño archivo: <tamaño en bytes>

- Tras iterar por todo el directorio, leerá un número de la STDIN. Si el número es 0, el programa mostrará por la STDOUT la suma total en bytes de todos los archivos que cumplieran la condición anterior, y finalizará.
- Adicionalmente, es muy recomendable comprobar los parámetros que se le pasa al programa, además de comprobar la ejecución de todas las llamadas al sistema (los if's)

Notas (no venían en el enunciado, pero son cosas que se deducen y hay que tener cuidado):

- Cuidado con la comparación de los permisos, cuando extraemos los atributos de un archivo en el struct de stat, el campo ***st_mode*** incluye tanto los permisos como el tipo de archivo, así que la comparación hay que hacerla sin el tipo de archivo:
 - `<atributos archivo>.st_mode & !permiso` : De esta forma le quitamos a los permisos el bit de tipo de archivo regular, quedando solo los permisos, y podremos comparar sin ningún tipo de problema.
- El profesor incidió mucho en el examen que el nº de bytes que leemos en la segunda lectura de la STDIN es 1, ya que solo leemos un 0 ("0"), mientras que en la primera lectura deben de leerse 3, un byte por cada dígito.

WUOLAH

Ejercicio 2 – distribuidor

Crear el programa *distribuidor*:

Sintaxis: distribuidor <dir> <n_hijos> <octal 1> <octal 2> ... <octal n_hijos>

Parámetros:

- <dir> Directorio donde queremos ejecutar el programa.
- <n_hijos> Número de hijos que queremos crear.
- <octal 1> Número octal que se le pasará al primer hijo para ejecutar.
- <octal 2> Número octal que se le pasará al segundo hijo para ejecutar.
- ...
- <octal n_hijos> Número octal que se le pasará al último hijo para ejecutar.

Funcionamiento:

- El programa creará tantos hijos como se especifique en el argumento, y cada uno de ellos ejecutará el programa *sumador* creado en el primer ejercicio, usando <dir> como argumento. Además, el programa se encargará de pasarle los octales que correspondan a cada hijo (el nº 1 al hijo 1, y así sucesivamente). Para pasar cada número, el programa padre empleará un cauce sin nombre para comunicarse con cada hijo.
- El padre, para indicar que ha terminado de mandar el número octal, deberá de escribir **tantos 0 como hijos creados**.
- Los hijos creados consumirán los octales, los cuales recibirán por el cauce desde su padre, y una vez ocurra eso, deberán enviarle a su padre el tamaño de los subtotales, a través de otro cauce sin nombre.
- Una vez haya consumido todos los valores enviados por los hijos, el proceso padre mostrará por STDOUT la suma de todos los subtotales recibidos.
- Es necesario realizar los direccionamientos pertinentes para que el padre mande octales a hijos y cada hijo el subtotal a su padre.

Notas:

Para aclarar un poco el funcionamiento del código (el enunciado lo copié un poco resumido, puede que haya perdido ciertos matices):

- El padre crea **2 cauces**, uno para enviar a todos sus hijos el octal que le corresponde, y otro para recibir el subtotal de sus hijos.
- Cada hijo debe cerrar los fd que hereda de su padre para poder establecer bien el flujo de información (el envío de información en un cauce es unidireccional, cuidado). El padre debe de cerrar los fd que debe para establecer bien el paso de información.
- Una vez establecido el flujo de información, es importante que los hijos redireccionen sus STDIN y STDOUT a los cauces para recibir información y mandar información a su padre de forma correcta.
- El padre manda a cada hijo a través del cauce el numero octal, y además un 0 para indicarle que ha finalizado de mandar el número octal y que puede enseñar el subtotal.

Examen SO Módulo II 2024

Ejercicio 1

```
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>
#include <errno.h>
#include <limits.h>

int main(int argc, char* argv[]){
    if( argc < 2){
        write(STDERR_FILENO, "./ej1 <dir>\n", 12);
        exit(1);
    }

    char *dir_name = argv[1];
    DIR *dir_ptr = opendir(dir_name);
    if( dir_ptr == NULL ){
        perror("opendir");
        exit(1);
    }

    int perm;

    // Leemos máscara octal
    if( scanf("%o", &perm) != 1 ){
        write(STDERR_FILENO, "Error leyendo mascara\n", 22);
        closedir(dir_ptr);
    }
}
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
exit(1);
}

while( perm != 0 ){

    long long tam_total = 0;

    rewinddir(dir_ptr); // Reseteamos el offset

    struct dirent *dir;
    while( (dir = readdir(dir_ptr)) != NULL ){

        char *name = dir->d_name;

        // Saltamos . y ..
        if( strcmp(name, ".") == 0 || strcmp(name, "..") == 0 )
            continue;

        char path[PATH_MAX];
        snprintf(path, sizeof(path), "%s/%s", dir_name, name);

        struct stat st;
        if( stat(path, &st) == -1 ){
            perror("stat");
            continue;
        }

        mode_t perms = st.st_mode & 0777;

        if( S_ISREG(st.st_mode) && perms == (mode_t)perm ){

            char buffer[256];
            int n = snprintf(buffer, sizeof(buffer),
                "Archivo: %s | Bloques: %lld | Tamaño archivo: %lld\n",
                name,
                (long long)st.st_blocks,
                (long long)st.st_size
            );
        }
    }
}
```

```

        if( n > 0 ){
            write(STDERR_FILENO, buffer, n);
        }

        tam_total += st.st_size;
    }
}

int x;
if( scanf("%d", &x) != 1 ){
    write(STDERR_FILENO, "Error leyendo numero final\n", 27);
    closedir(dir_ptr);
    exit(1);
}

if( x == 0 ){
    char out[64];
    int n = snprintf(out, sizeof(out), "%lld\n", tam_total);
    write(STDOUT_FILENO, out, n);
    closedir(dir_ptr);
    exit(0);
}

// Leemos nueva máscara si seguimos
if( scanf("%o", &perm) != 1 ){
    write(STDERR_FILENO, "Error leyendo mascara\n", 22);
    closedir(dir_ptr);
    exit(1);
}

closedir(dir_ptr);
return 0;
}

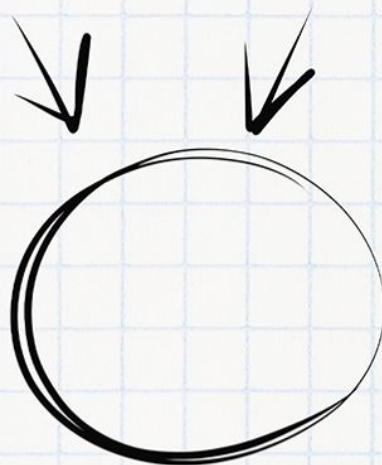
```


Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

Ejercicio 2

```
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>
#include <errno.h>
#include <limits.h>

int main(int argc, char *argv[]){
    if( argc < 4){
        fprintf(stderr, "./ej2 <dir> <n_hijos> <octal 1> ...\n");
        exit(1);
    }

    char *dir_name= argv[1];
    int n_hijos=atoi(argv[2]);

    if( n_hijos <= 0 ){
        fprintf(stderr, "n_hijos invalido\n");
        exit(1);
    }

    if( argc != 3 + n_hijos ){
        fprintf(stderr, "Error: numero de octales distinto a n_hijos\n");
        exit(1);
    }

    int entrada[2];
    if( pipe(entrada) == -1 ){
        perror("pipe entrada");
    }
}
```


Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
exit(1);
}

int salidas[2];
if( pipe(salidas) == -1 ){
    perror("pipe salidas");
    exit(1);
}

int octales[n_hijos];
int j=0;
for(int i=3; i < argc; i++){ //Guardamos todos los octales en un array
    // vienen como octal en texto → los pasamos a int (base 8)
    octales[j]=(int)strtol(argv[i], NULL, 8);
    j++;
}

//Creamos los hijos
int total=0;

for(int i=0; i < n_hijos; i++){
    pid_t pid=fork();
    if(pid == 0){

        // HIJO: solo lee de entrada[0] y escribe en salidas[1]
        close(entrada[1]); //Cerramos extremo de escritura
        close(salidas[0]); //Cerramos extremo de lectura

        dup2(entrada[0], STDIN_FILENO);
        dup2(salidas[1], STDOUT_FILENO);

        close(entrada[0]);
        close(salidas[1]);

        execlp("./sumador", "sumador", dir_name, (char*)NULL);

        perror("Exec");
        exit(1);
    }
}
```

```

    }

    else if(pid > 0){    //Padre

        // NO cerramos aqui entrada[0] ni salidas[1] porque los necesitan lo
        s futuros hijos

        // Enviamos al hijo su octal y un 0 (EN TEXTO, porque sumador lee c
        on scanf)
        char msg[64];

        int n = snprintf(msg, sizeof(msg), "%o\n", octales[i]);
        write(entrada[1], msg, n);

        write(entrada[1], "0\n", 2); // fin para ese hijo

        // Esperamos a que termine ese hijo
        waitpid(pid, NULL, 0);

        // Leemos su subtotal (sumador debe imprimir SOLO el numero por
        STDOUT)
        char buffer[128];
        int r = read(salidas[0], buffer, sizeof(buffer)-1);
        if(r <= 0){
            perror("read");
            exit(1);
        }
        buffer[r] = '\0';

        int recv = atoi(buffer);
        total += recv;
    }

    else {
        perror("fork");
        exit(1);
    }
}

```

```
// Cerramos pipes en el padre
close(entrada[0]);
close(entrada[1]);
close(salidas[0]);
close(salidas[1]);

printf("Tamaño total: %d\n", total);
return 0;
}
```